

Batch Processing with Job Scheduler

In this article, I will discuss **Batch Processing with a Job Scheduler** with one Real-time Example using EF Core Disconnected Entities. Please read our previous article discussing [Disconnected Entities in Entity Framework Core](#).

What is a Job Scheduler?

A Job Scheduler is a software application or service that automates the execution of tasks or jobs at specified times, intervals, or in response to specific conditions. These tasks can range from simple operations like running scripts or programs to more complex jobs like managing data backups or handling batch data processing and other scheduled activities. Job schedulers eliminate the need for manual intervention, making them essential in modern systems for ensuring periodic or automated task execution, such as processing payments, generating reports, or sending notifications.

Batch Processing Real-time Example using EF Core Disconnected Entities:

We need to develop one application that demonstrates a job scheduler designed to efficiently process pending payment statuses from an external payment gateway in batches. The job scheduler will retrieve payments with a "Pending" status and update their statuses based on the payment gateway's responses. It handles both success and failure scenarios. The implementation process includes three main components: Seeding Initial Data, Creating Jobs, and Processing Payments in batches.

- **Seeding Initial Data:** Set up initial sample data in the database.
- **Creating Jobs:** Create jobs that will be responsible for processing payments.
- **Processing Payments in Batches:** Retrieve and update payments in batches to ensure efficient processing and better performance.

Key Components

Entities:

- **Customer:** Represents a customer placing orders. Each customer can have multiple orders.
- **Order:** Represents the order made by the customer. Each order is associated with a single payment.
- **Payment:** Represents the payment associated with an order, including its current status (e.g., Pending, Completed, Failed).
- **Job:** Represents a job that processes the payments, tracking total payments, success/failure counts, and batch details.
- **JobDetail:** Represents details of each processed record within a job.

Batch Operations:

- **Batch Fetch:** Retrieve payments from the database with the "Pending" status.
- **Batch Update:** Update the status of each payment based on responses from the payment gateway.

Services:

- **Logger Service:** This service handles logging messages to a text file, providing clear audit information about the operations.
- **Job Service:** Manages job lifecycle operations, including job creation, completion, and logging of job details.
- **Payment Gateway Service:** This service simulates interaction with an external payment gateway, which returns the updated status of the payment.

- **Payment Service:** This service handles batch payment processing, interacting with other services to ensure smooth operations. It will use the payment gateway service to get updated payment statuses.

Let us proceed and implement the .NET Application first with the above requirement, and then we will discuss how to schedule the application using Windows Task Scheduler.

Customer Entity:

Create a class file named **Customer.cs** within the **Entities** folder, and then copy and paste the following code. This class represents a customer with a one-to-many relationship to orders.

```
namespace EFCoreCodeFirstDemo.Entities
{
    public class Customer
    {
        public int CustomerId { get; set; } // Primary Key
        public string Name { get; set; } // Customer name
        public string Email { get; set; } // Customer email
        // One-to-many relationship: A customer can have multiple orders
        public ICollection<Order> Orders { get; set; }
    }
}
```

Order Entity:

Create a class file named **Order.cs** within the Entities folder and then copy and paste the following code. It represents a customer's order and includes a one-to-one relationship with Payment

```
namespace EFCoreCodeFirstDemo.Entities
{
    public class Order
    {
        public int OrderId { get; set; } // Primary Key
        public DateTime OrderDate { get; set; } // Date of order placement
        public string Status { get; set; } // Status of the order (Pending,
Processing, Completed, Cancelled)
        public int CustomerId { get; set; } // Foreign Key to Customer
        public Customer Customer { get; set; } // Navigation property to
Customer
        // One-to-one relationship: Each order has a single payment.
        public Payment Payment { get; set; }
    }
}
```

Payment Entity:

Create a class file named **Payment.cs** within the Entities folder and then copy and paste the following code. This entity tracks payment details, including their status and any reasons for failure.

```
using System.ComponentModel.DataAnnotations.Schema;
namespace EFCoreCodeFirstDemo.Entities
```

```

{
    public class Payment
    {
        public int PaymentId { get; set; } // Primary Key
        [Column(TypeName = "decimal(18,2)")]
        public decimal Amount { get; set; } // Amount to be paid
        public string Currency { get; set; } // Currency type, e.g., USD
        public string Status { get; set; } // Payment status: Pending,
Completed, Failed, Cancelled
        public string TransactionId { get; set; } // External Transaction ID
        public string? FailureReason { get; set; } // Reason for failure, if
any
        public int OrderId { get; set; } // Foreign Key to Order
        public Order Order { get; set; } // Navigation property to Order
    }
}

```

Job Entity

Create a class file named **Job.cs** within the Entities folder, then copy and paste the following code. The Job entity tracks batch processing, including batch size, total payments, and success/failure counts.

```

namespace EFCoreCodeFirstDemo.Entities
{
    public class Job
    {
        public int JobId { get; set; } // Primary Key
        public DateTime StartTime { get; set; } // Job start time
        public DateTime? EndTime { get; set; } // Job end time
        public string Status { get; set; } // Job status: Started, Completed,
Failed, Partially Completed
        public int TotalPayments { get; set; } // Total number of payments in
the job
        public int SuccessfulPayments { get; set; } // Number of successful
payments
        public int FailedPayments { get; set; } // Number of failed payments
        public int BatchSize { get; set; } // Number of payments per batch
        public int TotalBatches { get; set; } // Total number of batches for
this job
        // Navigation property: A job can have many job details
        public ICollection<JobDetail> JobDetails { get; set; }
    }
}

```

JobDetail Entity

Create a class file named **JobDetail.cs** within the Entities folder, and then copy and paste the following code. Each job detail records the payment status change during the batch processing. Logs individual payment processing details for auditing purposes.

```

namespace EFCoreCodeFirstDemo.Entities

```

```

{
    public class JobDetail
    {
        public int JobDetailId { get; set; } // Primary Key
        public int JobId { get; set; } // Foreign Key to Job
        public int PaymentId { get; set; } // Foreign Key to Payment
        public string PreviousStatus { get; set; } // Previous payment status
        public string NewStatus { get; set; } // New payment status after
update
        public bool IsSuccess { get; set; } // Indicates whether the update
was successful
        public Job Job { get; set; } // Navigation property to Job
        public Payment Payment { get; set; } // Navigation property to
Payment
    }
}

```

Configuring the DbContext:

Modify the EFCoreDbContext class as follows:

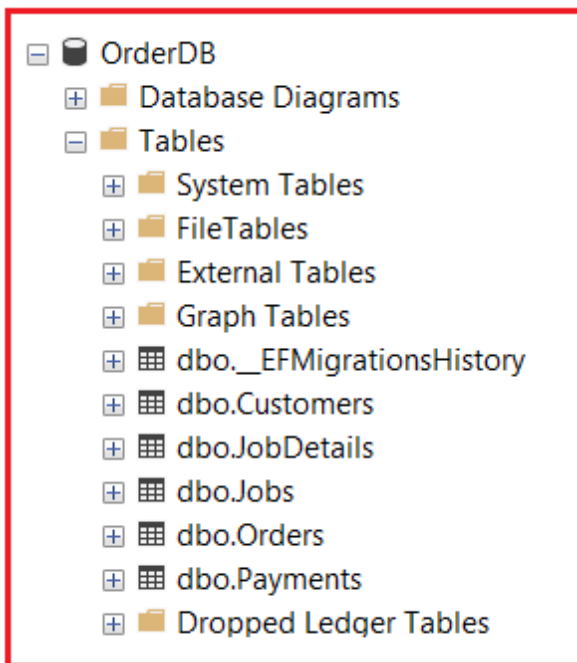
```

using Microsoft.EntityFrameworkCore;
namespace EFCoreCodeFirstDemo.Entities
{
    public class EFCoreDbContext : DbContext
    {
        protected override void OnConfiguring(DbContextOptionsBuilder optionsB
        {
            //Configuring the Connection String
            optionsBuilder.UseSqlServer(@"Server=LAPTOP-
6P5NK25R\SQLSERVER2022DEV;Database=OrderDB;Trusted_Connection=True;TrustServer
        }
        // DbSet
        public DbSet<Customer> Customers { get; set; }
        public DbSet<Order> Orders { get; set; }
        public DbSet<Payment> Payments { get; set; }
        public DbSet<Job> Jobs { get; set; }
        public DbSet<JobDetail> JobDetails { get; set; }
    }
}

```

Generating Migration and Updating the Database:

Open the Package Manager Console and Execute the **Add-Migration** and **Update-Database** commands. Once you execute these commands, the OrderDB database should be created with the required tables: Customers, Orders, Payments, Jobs, and JobDetails, as shown in the image below.



Implementing Services

We will create the following services:

- **Logger:** Handles logging messages to a text file.
- **JobService:** Manages job creation, completion, and logging job details.
- **PaymentGatewayService:** Simulates interactions with an external payment gateway.
- **PaymentService:** Handles batch operations related to payments.

We will create the above services inside the **Services** folder. First, create a folder called **Services** in the project root directory.

Logger Service:

Create a class file named **Logger.cs** within the services folder, and then copy and paste the following code. The following service handles logging messages to a text file. This implementation creates a daily log file and appends new log entries to it.

```
namespace EFCoreCodeFirstDemo.Services
{
    // Logger class to handle logging messages to a text file.
    public static class Logger
    {
        // Path to the log file
        private static string logFilePath;
        // Static constructor to ensure log directory exists
        static Logger()
        {
            // Define the folder path for logs
            string folderPath =
@"D:\EFCoreProjects\EFCoreCodeFirstDemo\EFCoreCodeFirstDemo\Logs"; // Update
this path as needed
        }
    }
}
```

```

        // Ensure the directory exists; if not, it will be created
        Directory.CreateDirectory(folderPath);
        // Get the current date and format it
        string currentDate = DateTime.Now.ToString("yyyyMMdd"); // e.g.,
20240922 for September 22, 2024
        // Define the file name with the current date
        string fileName = $"Log_{currentDate}.txt";
        // Combine the folder path and file name to create the full file
path
        logFilePath = Path.Combine(folderPath, fileName);
    }
    // Logs a message with a timestamp to the log file.
    public static void Log(string message)
    {
        try
        {
            // Prepare the log message with timestamp
            var logMessage = $"{DateTime.Now:yyyy-MM-dd HH:mm:ss} -
{message}";
            // Append the log message to the log file
            File.AppendAllText(logFilePath, logMessage +
Environment.NewLine);
        }
        catch (Exception ex)
        {
            // In case logging fails, log to console or a separate error
handling system.
            Console.WriteLine($"Logging failed: {ex.Message}");
        }
    }
}
}
}

```

Note: Consider using frameworks like NLog or Serilog for more robust logging. These frameworks offer advanced features such as asynchronous logging, multiple log targets, and structured logging.

Job Service

The Job Service will handle everything related to jobs, such as creating new jobs, updating job statuses, and recording job details (i.e., which payments were processed and their status changes). So, create a class file named **JobService.cs** within the Services folder and copy and paste the following code. This will manage creating and updating jobs, including logging job details.

```

using EFCoreCodeFirstDemo.Entities;
using Microsoft.EntityFrameworkCore;
namespace EFCoreCodeFirstDemo.Services
{
    public class JobService
    {
        // Creates a new job and stores it in the database.
        public async Task<Job> CreateNewJobAsync()
    }
}

```

```

{
    using var context = new EFCoreDbContext();
    var job = new Job
    {
        StartTime = DateTime.Now,
        Status = "Started",
        SuccessfulPayments = 0, // Initializing
        FailedPayments = 0,    // Initializing
        TotalPayments = 0,
        BatchSize = 0,
        TotalBatches = 0
    };
    context.Jobs.Add(job);
    await context.SaveChangesAsync();
    // Log job start
    Logger.Log($"Job {job.JobId} started at {job.StartTime}.");
    return job;
}
// Marks the job as completed and updates successful/failed payment
counts.
public async Task CompleteJobAsync(Job job)
{
    using var context = new EFCoreDbContext();
    job.EndTime = DateTime.Now;
    // Determine the job status based on the number of failed
payments
    if (job.FailedPayments > 0 && job.SuccessfulPayments > 0)
    {
        job.Status = "Partially Completed"; // Some payments failed
    }
    else if (job.FailedPayments == 0)
    {
        job.Status = "Completed"; // All payments succeeded
    }
    else
    {
        job.Status = "Failed"; // All payments failed
    }
    context.Entry(job).State = EntityState.Modified;
    await context.SaveChangesAsync();
    // Log job completion
    Logger.Log($"Job {job.JobId} completed at {job.EndTime}.
Successful payments: {job.SuccessfulPayments}, Failed payments:
{job.FailedPayments}.");
}
// Logs details of the payments processed by a job.
public async Task LogJobDetailsAsync(Job job, Payment payment, string
previousStatus, bool isSuccess)
{
    using var context = new EFCoreDbContext();
    var jobDetail = new JobDetail

```

```

    {
        JobId = job.JobId,
        PaymentId = payment.PaymentId,
        PreviousStatus = previousStatus,
        NewStatus = payment.Status,
        IsSuccess = isSuccess
    };
    context.JobDetails.Add(jobDetail);
    // Update job success/failure counters
    if (isSuccess)
    {
        job.SuccessfulPayments++;
    }
    else
    {
        job.FailedPayments++;
    }
    await context.SaveChangesAsync();
    // Log job detail
    Logger.Log($"Payment {payment.PaymentId}: Status updated from
    {previousStatus} to {payment.Status}. Success: {isSuccess}.");
}
}
}

```

Payment Gateway Service

This service simulates interaction with a payment gateway. It introduces random statuses and potential exceptions to simulate real-world errors. So, create a class file named **PaymentGatewayService.cs** within the Services folder and then copy and paste the following code:

```

using EFCoreCodeFirstDemo.Entities;
namespace EFCoreCodeFirstDemo.Services
{
    public class PaymentGatewayService
    {
        private readonly Random _random = new Random();
        private readonly List<string> _statuses = new List<string> {
            "Pending", "Completed", "Failed", "Cancelled" };
        // Simulates a network call to fetch payment status.
        public async Task<string> GetUpdatedPaymentStatusAsync(Payment
        payment)
        {
            try
            {
                // Randomly simulate network issues (e.g., gateway down).
                if (_random.Next(1, 10) > 8)
                {
                    throw new Exception("Payment gateway is temporarily
                    unavailable.");
                }
            }
        }
    }
}

```

```

        // Simulate network delay.
        await Task.Delay(200);
        // If the current status is "Pending", assign a new status.
        return payment.Status == "Pending" ?
        _statuses[_random.Next(_statuses.Count)] : payment.Status;
    }
    catch (Exception ex)
    {
        // Handle gateway error by throwing an exception with a
        specific message.
        throw new Exception($"Error accessing payment gateway for
        Payment ID {payment.PaymentId}: {ex.Message}");
    }
}
}
}

```

Payment Service

The Payment Service handles batch processing logic, interacting with the Job Service and Payment Gateway. All exceptions (such as gateway errors) are handled so they don't affect the rest of the batch. So, create a class file named **PaymentService.cs** within the Services folder and then copy and paste the following code:

```

using EFCoreCodeFirstDemo.Entities;
using Microsoft.EntityFrameworkCore;
namespace EFCoreCodeFirstDemo.Services
{
    public class PaymentService
    {
        private readonly PaymentGatewayService _paymentGatewaySimulator = new
        PaymentGatewayService();
        private readonly JobService _jobService = new JobService();
        // Processes pending payments in batches.
        public async Task ProcessPendingPaymentsAsync(Job job, int batchSize)
        {
            try
            {
                Logger.Log($"Starting batch processing for job
                {job.JobId}...");
                using var context = new EFCoreDbContext();
                // Fetch pending payments and initialize batch details.
                var pendingPayments = await context.Payments.AsNoTracking()
                .Where(p => p.Status == "Pending")
                .ToListAsync();
                job.TotalPayments = pendingPayments.Count;
                job.BatchSize = batchSize;
                job.TotalBatches =
                (int)Math.Ceiling((double)job.TotalPayments / batchSize);
                Logger.Log($"Total payments: {job.TotalPayments}. Total
                batches: {job.TotalBatches}.");
                // Process payments in batches.
            }
            catch (Exception ex)
            {
                // Handle gateway error by throwing an exception with a
                specific message.
                throw new Exception($"Error accessing payment gateway for
                Payment ID {job.PaymentId}: {ex.Message}");
            }
        }
    }
}

```

```

        for (int batchNumber = 1; batchNumber <= job.TotalBatches;
batchNumber++)
    {
        var currentBatch = pendingPayments
            .Skip((batchNumber - 1) * batchSize)
            .Take(batchSize)
            .ToList();
        if (!currentBatch.Any()) break;
        Logger.Log($"Processing Batch
{batchNumber}/{job.TotalBatches}...");
        using var updateContext = new EFCoreDbContext();
        foreach (var payment in currentBatch)
        {
            var previousStatus = payment.Status;
            var failureReason = string.Empty;
            var newStatus = "Pending";
            var isSuccess = false;
            try
            {
                // Fetch updated status from the payment gateway.
                newStatus = await
_paymentGatewaySimulator.GetUpdatedPaymentStatusAsync(payment);
                // If failed, store failure reason.
                if (newStatus == "Failed")
                {
                    failureReason = "Payment failed due to
gateway rejection.";
                }
                isSuccess = newStatus == "Completed";
            }
            catch (Exception ex)
            {
                Logger.Log($"Error for Payment ID
{payment.PaymentId}: {ex.Message}");
            }
            // Update payment details.
            payment.Status = newStatus;
            payment.FailureReason = failureReason;
            updateContext.Entry(payment).State =
EntityState.Modified;
            // Log job details.
            await _jobService.LogJobDetailsAsync(job, payment,
previousStatus, isSuccess);
        }
        // Save the current batch.
        await updateContext.SaveChangesAsync();
        Logger.Log($"Batch {batchNumber} processed
successfully.");
    }
    // Complete the job once all batches are processed.
    await _jobService.CompleteJobAsync(job);

```

```

    }
    catch (Exception ex)
    {
        Logger.Log($"Error processing payments: {ex.Message}");
    }
}
}
}

```

Modifying the Program Class:

Next, modify the Program class as follows:

```

using EFCoreCodeFirstDemo.Entities;
using EFCoreCodeFirstDemo.Services;
using Microsoft.EntityFrameworkCore;
namespace EFCoreCodeFirstDemo
{
    public class Program
    {
        private static readonly Random _random = new Random();
        static async Task Main(string[] args)
        {
            try
            {
                // Log application start
                Logger.Log("Job Scheduler Application Started.");
                // Seed the database with sample data if necessary
                await SeedDatabaseAsync();
                // Define batch size (number of payments to process per
batch)

                int batchSize = 20;
                // Initialize services
                var jobService = new JobService();
                var paymentService = new PaymentService();
                // Create a new job
                var job = await jobService.CreateNewJobAsync();
                // Start processing pending payments, passing the Job object
and batch size to PaymentService
                await paymentService.ProcessPendingPaymentsAsync(job,
batchSize);

                Logger.Log("Payment processing operations completed
successfully.");
            }
            catch (DbUpdateException ex)
            {
                Logger.Log($"Database Error Occurred:
{ex.InnerException?.Message ?? ex.Message}");
            }
            catch (Exception ex)
            {

```

```

        Logger.Log($"An unexpected error occurred: {ex.Message}");
    }
    finally
    {
        Logger.Log("Job Scheduler Application Ended.");
    }
}
static async Task SeedDatabaseAsync()
{
    using var context = new EFCoreDbContext();
    if (await context.Customers.AnyAsync())
    {
        Logger.Log("Database already contains data. Skipping
seeding.");
        return;
    }
    Logger.Log("Seeding database with sample data...");
    var customers = new List<Customer>();
    for (int i = 1; i <= 20; i++)
    {
        customers.Add(new Customer
        {
            Name = $"Customer_{i}",
            Email = $"customer{i}@example.com"
        });
    }
    context.Customers.AddRange(customers);
    await context.SaveChangesAsync();
    var orders = new List<Order>();
    for (int i = 1; i <= 100; i++)
    {
        var order = new Order
        {
            OrderDate = DateTime.Now.AddDays(-_random.Next(0, 30)),
            Status = "Pending",
            CustomerId =
customers[_random.Next(customers.Count)].CustomerId,
            Payment = new Payment
            {
                Amount = Math.Round((decimal)(_random.NextDouble() *
1000), 2),
                Currency = "USD",
                Status = "Pending",
                TransactionId = Guid.NewGuid().ToString()
            }
        };
        orders.Add(order);
    }
    context.Orders.AddRange(orders);
    await context.SaveChangesAsync();
    Logger.Log("Seeding completed.");
}

```

```
}  
}  
}
```

How Do We Create a Job Scheduler in Windows Operating System?

Creating a Job Scheduler in the Windows Operating System allows us to automate the execution of tasks, such as our .NET application for batch processing payment status updates. Windows offers built-in tools and mechanisms to schedule and manage jobs effectively. Let us understand how to create a job scheduler using Windows Task Scheduler and Windows Services.

Windows Task Scheduler

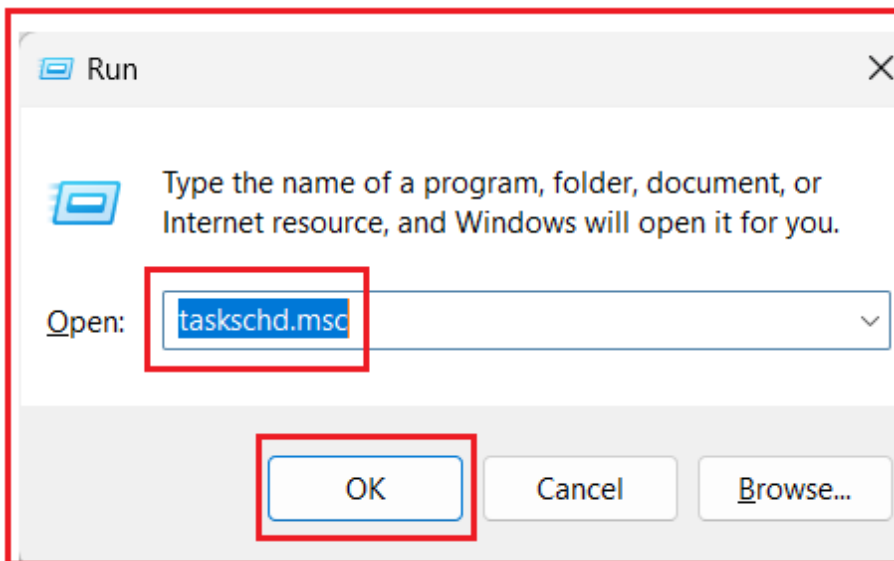
Windows Task Scheduler is a built-in Windows utility that allows us to automate the execution of scripts, programs, and other tasks at predefined times or in response to specific events. It's ideal for running periodic tasks without manual intervention.

Using Windows Task Scheduler to Schedule .NET Application

Windows Task Scheduler provides a user-friendly interface to schedule tasks. Let us understand the step-by-step process:

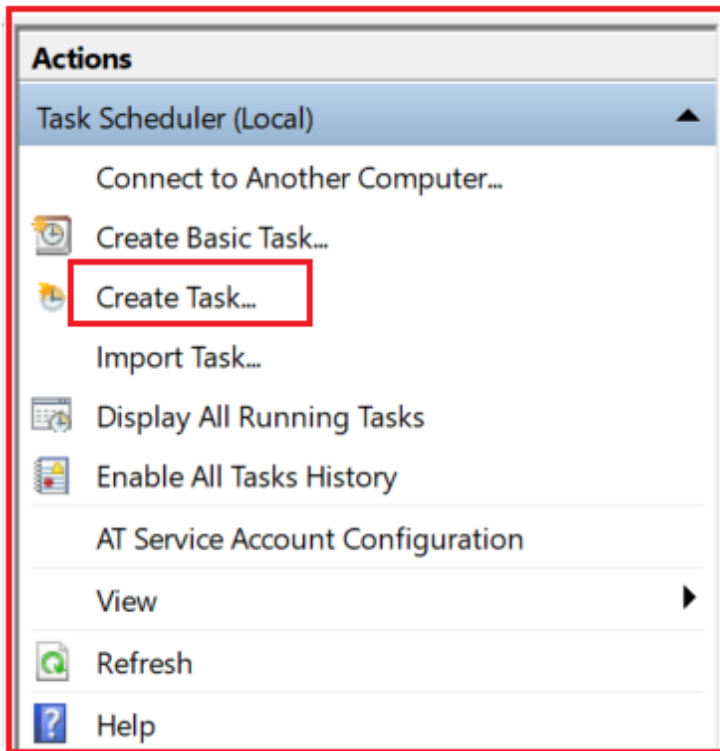
Step 1: Open Task Scheduler

Press Win + R, type taskschd.msc, and press Enter. Alternatively, search for "Task Scheduler" in the Start menu.



Step 2: Create a New Task

In the Task Scheduler window, click "Create Task..." in the Actions pane on the right.



Step 3: Configure the General Settings

Name: Provide a meaningful name, e.g., "PaymentStatusBatchProcessor".

Description: Optionally, add a description for clarity, such as "This Job will run every 30 minutes to process the Pending Payments in batches and update the status."

Security Options:

- Run whether the user is logged on or not: Allows the task to run in the background.
- Run with the highest privileges: If your application requires administrative rights.

Configure for: Select the appropriate Windows version.

Create Task

General Triggers Actions Conditions Settings

Name: PaymentStatusBatchProcessor

Location: \

Author: LAPTOP-6P5NK25R\Pranaya

Description: This Job will run in every 30 minutes to process the Pending Payments in batches to update the status

Security options

When running the task, use the following user account:

LAPTOP-6P5NK25R\Pranaya Change User or Group...

☒ Run only when user is logged on

☐ Run whether user is logged on or not

☐ Do not store password. The task will only have access to local computer resources.

☐ Run with highest privileges

☐ Hidden Configure for: Windows 10

OK Cancel

Step 4: Set Triggers

Navigate to the "Triggers" tab and click "New...".

Begin the task: Choose "On a schedule".

Settings: Define how often the task should run (daily, weekly, etc.).

Advanced Settings:

- **Repeat task every:** For recurring executions within a day.
- **Duration:** Set how long the repetition should continue.
- **Delay Task:** Add a delay before the task starts.

Click "OK" to save the trigger.

New Trigger ✕

Begin the task: On a schedule ▾

Settings

☐ One time Start: 22-09-2024 ▾ 11:30:24 ▾ ☐ Synchronize across time zones

☒ Daily

☐ Weekly

☐ Monthly

Recur every: 1 days

Advanced settings

☐ Delay task for up to (random delay): 1 hour ▾

☒ Repeat task every: 30 minutes ▾ for a duration of: 1 day ▾

☐ Stop all running tasks at end of repetition duration

☐ Stop task if it runs longer than: 3 days ▾

☐ Expire: 22-09-2025 ▾ 11:30:24 ▾ ☐ Synchronize across time zones

☒ Enabled

OK Cancel

Step 5: Define Actions

Go to the “Actions” tab and click “New...”.

- **Action:** Select “Start a program”.
- **Program/script:** Browse to the executable of your .NET application (e.g., D:\EFCoreProjects\EFCoreCodeFirstDemo\EFCoreCodeFirstDemo\bin\Debug\net8.0\EFCoreCodeFirstDemo.exe).
- **Add arguments (optional):** If your application accepts command-line arguments.
- **Start in (optional):** Specify the working directory if needed.

Click “OK” to save the action.

New Action

You must specify what action this task will perform.

Action: Start a program

Settings

Program/script:

D:\EFCoreProjects\EFCoreCodeFirstDemo\EFCoreCodeFirst Browse...

Add arguments (optional):

Start in (optional):

OK Cancel

Step 6: Configure Conditions and Settings (Optional)

Go to the Conditions Tab. Set conditions such as starting the task only if the computer is idle, on AC power, etc. We are going with default options.

The screenshot shows the 'Create Task' dialog box with the 'Conditions' tab selected. The dialog has five tabs: General, Triggers, Actions, Conditions, and Settings. The 'Conditions' tab contains instructions and several condition categories: Idle, Power, Wake the computer, and Network. The 'Power' section is highlighted with a red box, showing two checked options: 'Start the task only if the computer is on AC power' and 'Stop if the computer switches to battery power'. Other options include 'Start the task only if the computer is idle for' (with dropdowns for 10 minutes and 1 hour), 'Stop if the computer ceases to be idle' (checked), 'Restart if the idle state resumes' (unchecked), 'Wake the computer to run this task' (unchecked), and 'Start only if the following network connection is available' (unchecked, with a dropdown for 'Any connection').

Create Task

General Triggers Actions **Conditions** Settings

Specify the conditions that, along with the trigger, determine whether the task should run. The task will not run if any condition specified here is not true.

Idle

☐ Start the task only if the computer is idle for: 10 minutes

Wait for idle for: 1 hour

☒ Stop if the computer ceases to be idle

☐ Restart if the idle state resumes

Power

☒ Start the task only if the computer is on AC power

☒ Stop if the computer switches to battery power

☐ Wake the computer to run this task

Network

☐ Start only if the following network connection is available:

Any connection

Step 7: Settings Tab:

- Allow task to be run on demand
- Run task as soon as possible after a scheduled start is missed
- If the task fails, restart every: Configure retry attempts.
- Stop the task if it runs longer than: Prevent runaway tasks.

Click "OK" to create the task as shown in the below image:

Create Task

General Triggers Actions Conditions Settings

Specify additional settings that affect the behavior of the task.

☒ Allow task to be run on demand

☒ Run task as soon as possible after a scheduled start is missed

☐ If the task fails, restart every: 5 minute: ▾

Attempt to restart up to: 3 times

☒ Stop the task if it runs longer than: 3 days ▾

☒ If the running task does not end when requested, force it to stop

☐ If the task is not scheduled to run again, delete it after: 30 days ▾

If the task is already running, then the following rule applies:

Do not start a new instance ▾

OK Cancel

If prompted, enter the password for the user account under which the task will run.

Task Scheduler

Enter user account information for running this task.

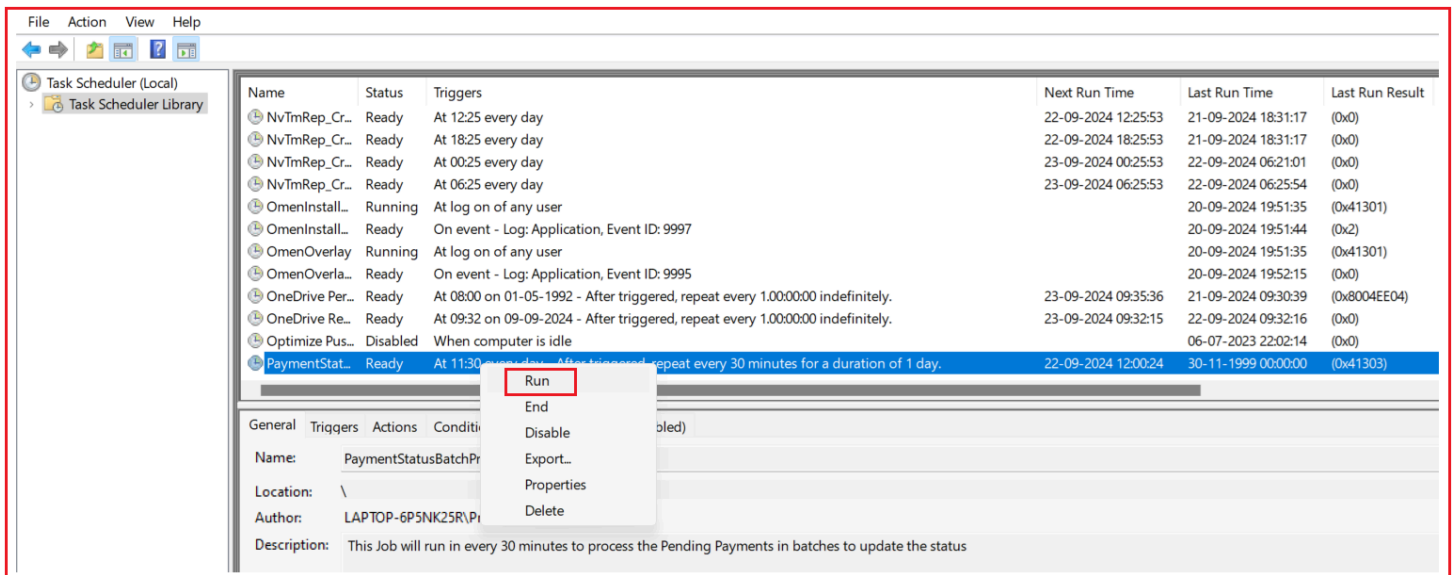
User name:  \PTOP-6P5NK25R\Pranaya ▾ ...

Password: ●●●●●●●●

OK Cancel

Step 8: Test the Scheduled Task

Locate your newly created task in the Task Scheduler Library. Right-click on the task and select “Run” to test it manually.



Verify that your application executes as expected and logs the appropriate entries. Now, please verify the log file and also verify the database tables. You should see that the application is working as expected. In our upcoming article, I will discuss creating a Background Server (worker process) using a .NET Application to schedule a job similar to Windows Task Scheduler.

When do we need a Job Scheduler?

Job schedulers are useful in scenarios such as:

- **Automating repetitive tasks:** This is when a task needs to be executed at regular intervals, such as database backups, file cleaning, report generation, etc.
- **Running tasks during off-hours:** Scheduling tasks during non-peak hours, such as running a report generation job at night.
- **Triggering long-running tasks:** Some processes, like large data processing jobs, take significant time and should be automated to start at a predefined time.
- **Batch Processing:** Automating batch operations such as billing processes, payment updates, etc.

In short, any situation where manual intervention is not feasible or efficient for repetitive or time-bound tasks would benefit from a job scheduler.

In the next article, I will discuss [Disconnected Entity Graph in Entity Framework Core](#) with Examples. Here, in this article, I explain Batch Processing with Job Scheduler with Examples. I hope you enjoyed this article on Batch Processing with Job Scheduler.